ELEC60004/70068. MACHINE REASONING

1. This question is about search space formalisation and the General Graph Search Engine (GGSE), and 'uninformed' search algorithms.

   The Amazing Word Machine (AMW) has a set of rules which transform one word into another. It uses the following rules:

   1. Add any vowel to the start of a word, i.e. $MAZE \rightarrow AMAZE$.
   2. Add any consonant to the end of a word, i.e. $CARD \rightarrow CARDS$.
   3. Delete two consecutive vowels, i.e. $PLEASE \rightarrow PLSE$.
   4. Delete two consecutive consonants, i.e. $STRING \rightarrow RING$, $TRING$ or $STRI$.
   5. Double the entire word. i.e. $AYE \rightarrow AYEAYE$.

   Starting with *LEAD*, you are asked to find a way to make *GOLD*.

   a) You are given two predicates `consonant/1` and `vowel/1`. What would be the expected difference between the queries:

   ```
   ?- vowel( X ).
   ```

   and

   ```
   ?- vowel( e ).
   ```

   [4]

   b) Using these two given predicates, formulate the state space so that it could be used with the GGSE:

   (*i*) Specify a state representation;
   (*ii*) Specify the initial state;
   (*iii*) Specify the final state; and
   (*iv*) Specify the state-change rules.

   [8]

   c) Briefly explain the difference between *breadth first*, *depth first*, *uniform cost* and *iterative deepening depth first* search algorithms.

   Using appropriate evaluation criteria, compare and contrast the four algorithms.

   Briefly comment on the suitability of each algorithm for solving word transformation problems with the AMW.

   [6]

   d) If Rule 3 and and Rule 4 were changed to delete *two or more* consecutive vowels or consonants (respectively), explain (declaratively) how to generate all solutions.

   [2]

(a)
Query with a variable would return all five vowels (i.e. succeed five times
Query with a constant term would return either true or false

```
/*
** (b)
** (i)
** List of letters
**
** (ii)
** initial state [l,e,a,d]
**
** (iii)
** goal state [g,o,l,d]
**
** (iv)
*/
state_change( rule1, Word, [Vowel|Word] ) :-
        vowel( Vowel ).

state_change( rule2, Word, NewWord ) :-
        consonant( C ),
        append( Word, [C], NewWord ).

state_change( rule3, Word, NewWord ) :-
        append( Front, [V1,V2|Back], NewWord ),
        vowel( V1 ),
        vowel( V2 ).

state_change( rule4, Word, NewWord ) :-
        append( Front, [C1,C2|Back], NewWord ),
        consonant( C1 ),
        consonant( C2 ).

state_change( rule5, Word, WordWord ) :-
        append( Word, Word, WordWord ).

/*
** (c)
*/
Breadth first: expand node on search frontier at minimum depth
Depth first: expand node on search frontier at maximum depth
Uniform Cost: expand node on search frontier with lowest cost from Start node
IDDF: depth first to successive values of depth d

Given:
        b = branching factor
        d = depth of solution
        m = maximum depth of tree
        w = width of the beam

                optimality      completeness    space-complex   time-complex
breadth         yes             yes             O(b^d)          O(b^d)
depth           no              no              O(b*m)          O(b^m)
uniform cost    yes             yes             O(b^d)          O(b^d)
IDDF            yes             yes             O(b*m)          O(b^d)

(With the usual caveats)

Breadth -- branching factor is huge is huge
Depth -- infinite loops e.g. rule 1
UC -- can't see what the cost is
```

```
IDDF -- Worst of both BF and DF worlds

/*
** (d)
*/
Check pairs at the head of the list
-- if they vowel-consonant or consonant-vowel, keep, cut and recurse on tail
-- if they are they same, delete successively longer consecutive vowels or
consonants until opposite is found
-- keep the head and recurse on the tail

This code is NOT required but this is one way to do it:

deln( [L1,L2|T1], [L1|T2] ) :-
        vowel( L1 ),
        consonant( L2 ), !,
        deln( [L2|T1], T2 ).
deln( [L1,L2|T1], [L1|T2] ) :-
        consonant( L1 ),
        vowel( L2 ), !,
        deln( [L2|T1], T2 ).
deln( [H|T], Result ) :-
        delete_same( H, T, Result ).
deln( [H|T], [H|Result]  ) :-
        deln( T, Result ).

delete_same( L1, [L2|_], _ ) :-
        vowel( L1 ),
        consonant( L2 ),
        !, fail.
delete_same( L1, [L2|_], _ ) :-
        consonant( L1 ),
        vowel( L2 ),
        !, fail.
delete_same( _, [_|Result], Result ).
delete_same( _, [L2|T], Result ) :-
        delete_same( L2, T, Result ).

Before calling deln need to check for empty list or singleton list.

Gives

| ?- deln( [s,t,r,i,i,i,n,g], X ).

X = [r, i, i, i, n, g];

X = [i, i, i, n, g];

X = [s, i, i, i, n, g];

X = [s, t, r, i, n, g];

X = [s, t, r, n, g];

X = [s, t, r, i, n, g];

X = [s, t, r, i, i, i];

no
```

2. This question is about search space formalisation and the General Graph Search Engine (GGSE), and 'informed' search algorithms.

Two adults and two children need to cross a river in a rowing boat, subject to the following conditions:

1. All adults weigh the same.
2. Each child weighs half as much as an adult.
3. The boat can only carry the weight equivalent of one adult.
4. Adults and children can row the boat.
5. The boat must have someone in it to row it.

The problem is to get everyone across the river, given that they all start on the left bank (with the boat).

a) Specify a state representation, and justify your state representation with respect to alternative representations for river-crossing problems of this kind.

[4]

b) Given this state representation, formulate the state space so that it could be used with the GGSE:

(*i*) Specify the initial state;
(*ii*) Specify the final state; and
(*iii*) Specify the state-change rules.

[6]

c) With a definition of a graph $G = \langle N, E, R \rangle$ and an identified node $S \in N$, give an inductive definition for the set of paths P_{G_S} rooted on S.

[3]

d) With a definition of a graph $G' = \langle S, Op \rangle$, give an inductive definition for $P_{G'}$, the set of paths defined by G'.

[3]

e) Explain how to show that $P_{G_S} = P_{G'}$.

[2]

f) Explain how the paths in the inductive definition(s) are searched by the A* search algorithm.

[2]

```
/*
** (a)
** 2-tuple (people(left(AL,CL),right(AR,CR)), boat(Bank))
** where AL is number adults on left, AR is number of adults on right etc
**
** They are undifferentiated mass in this problem
** Alternatives are to represent them as individuals, but they are fungible
** and have no uniquely identifying properties as in the bridge-torch;
** or to represent them by position on river, but they have no inter-dependent
** relations, and this does not scale
**
*/

/*
** (b)
** (i)
** initial state (people(left(2,2),right(0,0)),boat(left))
**
** (ii)
** goal state (people(left(0,0),right(2,2)),boat(right))
**
** (iii)
** state_change rules:
*/
state_change( adult, (people(left(AL,CL),right(AR,CR)), boat(Bank)),
                     (people(left(NewAL,CL),right(NewAR,CR)), boat(Opposite)) :-
        move_adult( Bank, AL, AR, Opposite, NewAL, NewAR ).

state_change( child1, (people(left(AL,CL),right(AR,CR)), boat(Bank)),
                      (people(left(AL,NewCL),right(AR,NewCR)), boat(Opposite)) :-
        move_1_child( Bank, CL, CR, Opposite, NewCL, NewCR ).

state_change( child2, (people(left(AL,CL),right(AR,CR)), boat(Bank)),
                      (people(left(AL,NewCL),right(AR,NewCR)), boat(Opposite)) :-
        move_2_child( Bank, CL, CR, Opposite, NewCL, NewCR ).


move_adult(left, AL, AR, right, NewAL, NewAR ) :-
        AL > 0, !,
        NewAL is AL - 1,
        NewAR is AR + 1.
move_adult(right, AL, AR, left, NewAL, NewAR ) :-
        AR > 0, !,
        NewAL is AL + 1,
        NewAR is AR - 1.

move_1_child( left, CL, CR, right, NewCL, NewCR ) :-
        CL > 0, !,
        NewCL is CL - 1,
        NewCR is CR + 1.
move_1_child( right, CL, CR, left, NewCL, NewCR ) :-
        CR > 0, !,
        NewCR is CR - 1,
        NewCL is CL + 1.

move_2_child( left, 2, 0, right, 0, 2 ) :-
        !.
move_2_child( right, 0, 2, left, 2, 0 ) :-
        !.
```

/* ** (c) */

$$P_G = \bigcup_{i=0}^{\infty} P_i$$
$$P_0 = \{[S]\}$$
$$P_{i+1} = \{[n \mid p] \mid \exists p \in P_i.(frontier(p), e, n) \in R\}$$

/* ** (d) */

$$P_{G'} = \bigcup_{i=0}^{\infty} P'_i$$
$$P'_0 = \{[S]\}$$
$$P'_{i+1} = \{[n \mid p] \mid \exists p \in P'_i.\exists op \in Op.op(frontier(p)) = (n, e)\}$$

/* ** (e) */ Given the condition

$$(n, e, n') \in R \leftrightarrow \exists op_i \in Op.op_i(n) = (n', e)$$

Prove by induction ion the lengths of the path

/* ** (f) */

Expands that path in whichever P_i has been partially or completely filled in, whose frontier node's h-cost is least

3. This question is about knowledge representation and reasoning with Predicate Logic, Prolog, and **KE** for Predicate Logic.

Important: any answer to any part that doesn't use line numbers and rule references to indicate how new formulas have been derived will not get any marks.

a) Consider the following English statements:

If anyone chooses rock and anyone chooses scissors, then whoever chose rock beats whoever chose scissors.
If anyone chooses scissors and anyone chooses paper, then whoever chose scissors beats whoever chose paper.
If anyone chooses paper and anyone chooses rock, then whoever chose paper beats whoever chose rock.
The 'beats' relation is transitive.
Alana chooses rock.
Beatrice chooses scissors.
Charlotte chooses paper.

Express these statements in first-order predicate logic, and convert them to a set of Horn clauses.

[4]

b) Prove, using resolution and proof by negation, that *Alana beats Charlotte*. The proof should show all the unifiers.

[4]

c) Hence, or otherwise, express the Horn Clauses of part (a) as a Prolog program, and briefly explain whether or not the query `?- beats( alana, alana ).` succeeds.

[4]

d) Prove the following argument the **KE** calculus for Predicate Logic:

No mammals are frogs.
All elephants are mammals.
Therefore, no elephants are frogs.

[4]

e) Using **KE** calculus for Predicate Logic, prove that the following argument is fallacious:

Some trees are tall things.
Some tall things are buildings.
Therefore, some trees are buildings.

[4]

** (a) */

In Predicate Logic

- $\forall x.\forall y.(chooses(x, rock) \wedge chooses(y, scissors)) \rightarrow beats(x, y)$
- $\forall x.\forall y.(chooses(x, scissors) \wedge chooses(y, paper)) \rightarrow beats(x, y)$
- $\forall x.\forall y.(chooses(x, paper) \wedge chooses(y, rock)) \rightarrow beats(x, y)$
- $\forall x.\forall y.(beats(x, y) \wedge beats(y, z)) \rightarrow beats(x, z)$
- $chooses(alana, rock)$
- $chooses(beatrice, scissors)$
- $chooses(charlotte, paper)$

As Horn Clauses:

$\neg chooses(X1, rock) \vee \neg chooses(Y1, scissors) \vee beats(X1, Y1)$	p1
$\neg chooses(X2, scissors) \vee \neg chooses(Y2, paper) \vee beats(X2, Y2)$	p2
$\neg chooses(X3, paper) \vee \neg chooses(Y3, rock) \vee beats(X3, Y3)$	p3
$\neg beats(X4, Y4) \vee \neg beats(Y4, Z4) \vee beats(X4, Y4)$	p4
$chooses(alana, rock)$	p5
$chooses(beatrice, scissors)$	p6
$chooses(charlotte, paper)$	p7

(b)

	$\neg beats(alana, charlotte)$		$\neg$conc
1	$\neg beats(alana, Y4) \vee \neg beats(Y4, charlotte)$	$\{X4 \mapsto alana, Z4 \mapsto charlotte\}$	($\neg$conc, p4)
2	$\neg chooses(alana, rock) \vee \neg chooses(Y1, scissors) \vee \neg beats(Y1, charlotte)$	$\{Y4 \mapsto Y1\}$	(1, p1)
3	$\neg chooses(Y1, scissors) \vee \neg beats(Y1, charlotte)$		(2, p5)
4	$\neg beats(beatrice, charlotte)$	$\{Y1 \mapsto beatrice\}$	(3, p6)
5	$\neg chooses(beatrice, scissors) \vee \neg chooses(charlotte, paper)$	$\{X2 \mapsto beatrice, Y2 \mapsto charlotte\}$	(4, p2)
6	$\neg chooses(charlotte, paper)$		(5, p6)
7	$\bot$		(6, p7)

/*

(c)

```
beats( X, Y ) :-
        chooses( X, rock ),
        chooses( Y, scissors ).

beats( X, Y ) :-
        chooses( X, scissors ),
        chooses( Y, paper ).

beats( X, Y ) :-
        chooses( X, paper ),
        chooses( Y, rock ).

beats( X, Z ) :-
        beats( X, Y ),
        beats( Y, Z ).

chooses( alana, rock ).
chooses( beatrice, scissors ).
chooses( charlotte, paper ).
```

The third clause, not used in the proof of part (b), allows us to prove that alana beats herself.

/* ** (d) */

1	$\neg\exists x.mammal(x) \wedge frog(x)$	premise
2	$\forall x.elephant(x) \rightarrow mammal(x)$	premise
3	$\neg(\neg\exists x.elephant(x) \wedge frog(x))$	$\neg$ conc
4	$\exists x.elephant(x) \wedge frog(x)$	$\neg\neg$, 3
5	$elephant(k) \wedge frog(k)$	$\exists$, 4
6	$elephant(k)$	α, 5
7	$frog(k)$	α, 5
8	$elephant(k) \rightarrow mammal(k)$	$\forall$, 2
9	$mammal(k)$	β, 8, 6
10	$\neg(mammal(k) \wedge frog(k))$	$\forall$, 1
11	$\neg frog(k)$	β, 10, 9
	close, 7, 11	

/* ** (e) */

1	$\exists x.tree(x) \wedge tallthing(x)$	premise
2	$\exists x.tallthing(x) \wedge building(x)$	premise
3	$\neg(\exists x.tree(x) \wedge building(x))$	$\neg$ conc
4	$tree(k1) \wedge tallthing(k1)$	$\exists$, 1
5	$tree(k1)$	α, 4
6	$tallthing(k1)$	α, 4
7	$tallthing(k2) \wedge building(k2)$	$\exists$, 2
8	$tallthing(k2)$	α, 7
9	$building(k2)$	α, 7
10	$\neg(tree(k1) \wedge building(k1))$	$\forall$, 3
11	$\neg(tree(k2) \wedge building(k2))$	$\forall$, 3
12	$\neg building(k1)$	β, 10, 5
13	$\neg tree(k2)$	β, 11, 9
open		

Build a model which makes the premises true, and the conclusion false.

So $k1$ is a tree and a tallthing, but not a building; and $k2$ is a tallthing and a building, but not a tree.

4. This question is about reasoning in Propositional Logic and Propositional Modal Logic, using the **KE** calculus.

Important: any answer to any part that doesn't use line numbers and rule references to indicate how new formulas have been derived will not get any marks.

a) Prove, using the the **KE** calculus, that the following are theorems of Propositional Logic:

(*i*) $(p \rightarrow (q \rightarrow r)) \rightarrow (\neg r \rightarrow (p \rightarrow -q))$
(*ii*) $(p \rightarrow q) \rightarrow ((r \rightarrow \neg q) \rightarrow (p \rightarrow \neg r))$

[4]

b) In the *Obligatio Game*, a finite number n of rounds is chosen, depending on the severity of the exam. The examiner then gives the candidate successive assertions, $\phi_1, \phi_2, \ldots, \phi_n$ that she or he has to either *accept* or *reject* as each one is put forward. In the former case, ϕ_i is added to the candidate's stock of commitments; in the latter case, the negation $\neg\phi_i$ is added.

The candidate passes the exam if she maintains the consistency of her stock of commitments throughout all n assertions.

Suppose the candidate is exposed (successively) to the following three ($n = 3$) assertions:

1. $p \vee \neg r$
2. $\neg(q \wedge r)$
3. $q \leftrightarrow r$

Using the **KE** calculus, show what sequences of answers would ensure that the student passes the exam.

(Hint: use **KE** for model building, for all combinations of *accept* or *reject* of the assertions. If the proof has at least one open branch, then this implies a consistent sequence of answers.)

[8]

c) Define a Kripke Model, and explain how it is used to give a semantics for formulas of Modal Logic.

[4]

d) Explain how $\models^{\mathscr{M}}_{\alpha} \Box p$ can be true in a model $\mathscr{M}$ while $\models^{\mathscr{M}}_{\alpha} \Diamond p$ can be false in the same model.

Explain how this is not the case if the accessibility relation satisfies the serial condition.

Using the **KE** calculus for Modal Logic, prove that the characteristic formula for the serial condition, $\Box P \rightarrow \Diamond P$, is a theorem of modal logic **S5**.

[4]

```
(a)
(i)
| ?- ke( [ -( (p->(q->r)) -> (-r -> (p-> -q)) ) ] ).
   1     - ((p -> q -> r) -> - r -> p -> - q)
   2     alpha 1         p -> q -> r
   3     alpha 1         - (- r -> p -> - q)
   4     alpha 3         - r
   5     alpha 3         - (p -> - q)
   6     alpha 5         p
   7     alpha 5         - (- q)
   8     dne   7         q
   9     beta  2,6       q -> r
  10     beta  9,8       r
         close 10,4

(ii)
| ?- ke( [ -( (p->q) -> ((r-> -q) -> (p-> -r)) ) ] ).
   1     - ((p -> q) -> (r -> - q) -> p -> - r)
   2     alpha 1         p -> q
   3     alpha 1         - ((r -> - q) -> p -> - r)
   4     alpha 3         r -> - q
   5     alpha 3         - (p -> - r)
   6     alpha 5         p
   7     alpha 5         - (- r)
   8     dne   7         r
   9     beta  2,6       q
  10     beta  4,8       - q
         close 10,9

(b)
[p + - r, - (q & r), q <-> r]
   1     p + - r
   2     - (q & r)
   3     q <-> r
   4     br-1  3              q
   5          eta   3,4       r
   6          beta  2,4       - r
              close 6,5
   7     br-2  3              - q
   8          eta   3,7       - r
              open [- q, - r]

[p + - r, - (q & r), - (q <-> r)]
   1     p + - r
   2     - (q & r)
   3     - (q <-> r)
   4     br-1  3              q
   5          eta   3,4       - r
              open [q, - r]
   6     br-2  3              - q
   7          eta   3,6       r
   8          beta  1,7       p
              open [p, r, - q]

[p + - r, - (- (q & r)), q <-> r]
   1     p + - r
   2     - (- (q & r))
   3     q <-> r
   4     dne   2         q & r
   5     alpha 4         q
   6     alpha 4         r
   7     eta   3,5       r
```

```
   8     beta  1,7       p
         open [p, q, r]

[p + - r, - (- (q & r)), - (q <-> r)]
   1     p + - r
   2     - (- (q & r))
   3     - (q <-> r)
   4     dne   2         q & r
   5     alpha 4         q
   6     alpha 4         r
   7     eta   3,5       - r
         close 7,6

[- (p + - r), - (q & r), q <-> r]
   1     - (p + - r)
   2     - (q & r)
   3     q <-> r
   4     alpha 1         - p
   5     alpha 1         - (- r)
   6     dne   5         r
   7     beta  2,6       - q
   8     eta   3,7       - r
         close 8,6

[- (p + - r), - (q & r), - (q <-> r)]
   1     - (p + - r)
   2     - (q & r)
   3     - (q <-> r)
   4     alpha 1         - p
   5     alpha 1         - (- r)
   6     dne   5         r
   7     beta  2,6       - q
   8     eta   3,7       r
         open [r, - p, - q]

[- (p + - r), - (- (q & r)), q <-> r]
   1     - (p + - r)
   2     - (- (q & r))
   3     q <-> r
   4     alpha 1         - p
   5     alpha 1         - (- r)
   6     dne   5         r
   7     dne   2         q & r
   8     alpha 7         q
   9     alpha 7         r
  10     eta   3,8       r
         open [q, r, - p]

[- (p + - r), - (- (q & r)), - (q <-> r)]
   1     - (p + - r)
   2     - (- (q & r))
   3     - (q <-> r)
   4     alpha 1         - p
   5     alpha 1         - (- r)
   6     dne   5         r
   7     dne   2         q & r
   8     alpha 7         q
   9     alpha 7         r
  10     eta   3,8       - r
         close 10,9

So

yes, yes, yes
yes, yes, no
```

```
yes, no, yes
no, yes, no
no, no, yes
```

(c)

$\mathscr{M} = \langle W, R, [\![\cdot]\!] \rangle$

where

W is a non-empty set of worlds, R is the accessibility relation on worlds, and $[\![\cdot]\!]$ is the denotation function which maps propositions to worlds in which they are true.

$$\models^{\mathscr{M}}_{\alpha} \Box p \quad \textit{iff} \quad \forall \beta \in \mathscr{M}.\alpha R \beta \rightarrow \models^{\mathscr{M}}_{\beta} p$$
$$\models^{\mathscr{M}}_{\alpha} \Diamond p \quad \textit{iff} \quad \exists \beta \in \mathscr{M}.\alpha R \beta \wedge \models^{\mathscr{M}}_{\beta} p$$

(d)

The semantics of $\Box$ is "for any world, if accessible then …". If no world is accessible then the antecedent of the implication is always false, and false *implies* anything is true.

The semantics of $\Diamond$ is "for some world, if accessible and …". If no world is accessible then first component of the and is always false, and false *and* anything is false.

(e)

The serial condition states that $\forall \alpha \in R.\exists \beta \in R.\alpha R \beta$.

Therefore for any world, there must be some world that is accessible. Therefore we cannot have $\Box p$ true and $\Diamond p$ false, which would contradict this characteristic formula.

The proof in S5 is trivial

1	[1]:	$\neg(\Box p \rightarrow \Diamond p)$	$\neg conc$
2	[1]:	$\Box p$	$\alpha, 1$
3	[1]:	$\neg\Diamond p$	$\alpha, 1$
4	[1];	p	$\Box$ (in S5, can access any available world on the branch)
5	[1];	$\neg p$	$\Box$ ditto

close 4, 5

5. This question is about the Simplified Event Calculus (SEC).

Consider the Glass-Filling Problem of having two glasses, one that can hold seven pints and one that can hold five pints, and the problem of getting four pints in the seven-pint glass.

a) Represent this problem in the SEC, specifying:
   (*i*) the multi-valued fluents;
   (*ii*) the `initially` facts; and
   (*iii*) `initiates` axioms for the *fill*, *empty* and either one of the *pour* (from one glass to another) events.

[12]

b) Give an example narrative, and explain how the SEC can be used to implement problem-solving by *planning* (i.e. generate a narrative (plan) that solves the problem).

[4]

c) Explain, with the use of the Glass-Filling Problem as an example, how knowledge is represented and reasoned with in the Simplified Event Calculus compared to the state-space formulation as a graph, and briefly discuss the implications for planning to solve problems of this kind.

[4]

```
(a)
/*
** Multi-valued Fluents are
** five, which we suppose can take an integer value in [0..5]
** seven, we suppose can take an integer value in [0..7]
*/

/*
** Initially, both glasses are empty
*/
initially( five=0 ).
initially( seven=0 ).

/*
** Initiates (this is for all events, only one pour event is required
** because they are symmetric
** but note that we need four axioms:
**   * if the combined value is greater than the limit of the glass being poured into
**     - new value initiated for five glass
**     - new value initiated for seven glass
**   * if the combined value is less than the limit of the glass being poured into
**     - new value initiated for five glass
**     - new value initiated for seven glass
*/
initiates( fill5, five=5, _ ).

initiates( fill7, seven=7, _ ).

initiates( empty5, five=0, _ ).

initiates( empty7, seven=0, _ ).

initiates( pour5to7, seven=7, T ) :-
        holdsAt( five=F, T ),
        F > 0,
        holdsAt( seven=S, T ),
        S < 7,
        FS is F + S,
        FS >= 7.
initiates( pour5to7, five=Frem, T ) :-
        holdsAt( five=F, T ),
        F > 0,
        holdsAt( seven=S, T ),
        S < 7,
        FS is F + S,
        FS >= 7,
        Frem is FS - 7.
initiates( pour5to7, seven=FS, T ) :-
        holdsAt( five=F, T ),
        F > 0,
        holdsAt( seven=S, T ),
        S < 7,
        FS is F + S,
        FS < 7.
initiates( pour5to7, five=0, T ) :-
        holdsAt( five=F, T ),
        F > 0,
        holdsAt( seven=S, T ),
        S < 7,
        FS is F + S,
        FS < 7.
```

```
initiates( pour7to5, seven=Srem, T ) :-
        holdsAt( five=F, T ),
        F < 5,
        holdsAt( seven=S, T ),
        S > 0,
        FS is F + S,
        FS >= 5,
        Srem is FS - 5.
initiates( pour7to5, five=5, T ) :-
        holdsAt( five=F, T ),
        F < 5,
        holdsAt( seven=S, T ),
        S > 0,
        FS is F + S,
        FS >= 5.
initiates( pour7to5, seven=0, T ) :-
        holdsAt( five=F, T ),
        F < 5,
        holdsAt( seven=S, T ),
        S > 0,
        FS is F + S,
        FS < 5.
initiates( pour7to5, five=FS, T ) :-
        holdsAt( five=F, T ),
        F < 5,
        holdsAt( seven=S, T ),
        S > 0,
        FS is F + S,
        FS < 5.


(b)

happens( fill7, 1 ).
happens( pour7to5, 2 ).
happens( empty5, 3 ).
happens( pour7to5, 4 ).
happens( fill7, 5 ).
happens( pour7to5, 6 ).

\begin{itemize}
        \item Assume: perform one action at each consecutive time point
        \item At each time point, select a possible action, add (assert) to database
        \item If it leads to a consistent narrative, create history and try another
        \item If if leads to a 'dead end', retract, backtrack and try another
\end{itemize}

(c)

statechange( fill5, state(F,S), state(5,S) ).
** etc.

With state space formulation, we have a single global state, whereas in the EC,
each fluent represents a local state. We can reconstruct the global state by asking
what \texttt{holdsAt} any time point.

Then to apply a state-change rules, we have to explicitly list everything in the state,
including those things that are not changed. With EC, we only have to specify the axioms
for those fluents whose values are changed by the corresponding event, everything else
stays the same (common sense law of inertia).

This can be a limitation as the state-space scales, we have to track more and more parameter
in the state space. The limitation of the EC is the amount of recursion that can cause problems
as the narrative gets longer, and for example there is no caching going on.
```